

Programming Language Principles, Design, and Implementation

Compilers - Lexical Analysis

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**

Today

Lexical Analysis

- ▶ Regular expressions
- ▶ Finite Automata
- ▶ Regular expressions to NFAs
- ▶ NFAs to DFAs
- ▶ Building a lexical analyzer

Further reading:

- ▶ Chapter 2 of “Modern Compiler Implementation”
- ▶ Chapter 3 of the Dragon book

Problem

- ▶ First phase of a compiler
- ▶ **Goal:**
 - ▶ **input:** the characters of the source program
 - ▶ **output:** a sequence of **tokens**
- ▶ It splits the input into a sequence of lexemes, from which it extracts tokens
- ▶ Interacts with the symbol table to store **lexemes**
- ▶ Commonly called by the parser to compute the next token

Concepts:

- ▶ **Lexeme:** sequence of characters
- ▶ **Token:** a token name sometimes accompanied by an attribute value
- ▶ **Symbol table:** datastructure storing information about the symbols (e.g., names and types of identifiers)

Example

Consider the expression: `if  $i = 0$  then true else false`

A lexer converts it into

IF, $\langle \text{ID}, 1 \rangle$, **EQ**, $\langle \text{INT}, 0 \rangle$, **THEN**, **TRUE**, **ELSE**, **FALSE**

Lexemes & tokens:

Lexeme	Token
if	IF
i	$\langle \text{ID}, 1 \rangle$
=	EQ
0	$\langle \text{INT}, 0 \rangle$
then	THEN
true	TRUE
else	ELSE
false	FALSE

The token names: **IF**, **ID**, **EQ**, **INT**, **THEN**, **TRUE**, **ELSE**, **FALSE** are abstract symbols

The token $\langle \text{ID}, 1 \rangle$ points to the 1st entry in the symbol table:

1	i	...
...		

Regular Expressions

How can we specify all the lexemes that can occur in a program?

We can use **regular expressions** (REs)

Given an alphabet Σ , a RE e is either

- ▶ a symbol $a \in \Sigma$
- ▶ the empty string ϵ
- ▶ an alternation/union $e_1 \mid e_2$
- ▶ a concatenation $e_1 e_2$
- ▶ a repetition e^*

The language $L(e)$ captured by the RE e is a set of words (lexemes):

- ▶ $L(a) = \{a\}$, where $a \in \Sigma$
- ▶ $L(\epsilon) = \{\epsilon\}$
- ▶ $L(e_1 \mid e_2) = L(e_1) \cup L(e_2)$
- ▶ $L(e_1 e_2) = \{w_1 w_2 \mid w_1 \in L(e_1) \wedge w_2 \in L(e_2)\}$
- ▶ $L(e^*) = \bigcup_{n \in \mathbb{N}} L(e^n)$, where $L(e^0) = \{\epsilon\}$ and $L(e^{n+1}) = L(e e^n)$

Regular Expressions

What are the languages captured by the following regular expressions

$a \mid b$
 $(a \mid b)^*$
 $a(aa)^*$

- ▶ $a \mid b$: $\{a, b\}$
- ▶ $(a \mid b)^*$: $\{\epsilon, a, b, aa, ab, ba, bb, \dots\}$
- ▶ $a(aa)^*$: $\{a, aaa, aaaaaa, \dots\}$

Regular Expressions

Abbreviations:

- ▶ We write $[a-z]$ for $a \mid \dots \mid z$ and $[0-9]$ for $0 \mid \dots \mid 9$
- ▶ We write e^+ for ee^*
- ▶ We write $e?$ for $e|\epsilon$

Examples:

- ▶ The words starting with a's followed by b's: a^*b^*
- ▶ identifiers could be: $[a-z][a-z]^*[0-9]^*$

Finite Automata

Regular expressions are not convenient to compute whether a word is part of language

Instead, we will use **Finite Automata** (FA)

A finite automata is defined as a tuple: $M = \langle Q, \Sigma, \delta, q_0, F \rangle$ where

- ▶ Q : set of states
- ▶ Σ : alphabet
- ▶ δ : transition relation
- ▶ $q_0 \in Q$: start state
- ▶ $F \subseteq Q$: set of final states

Deterministic Finite Automata (DFA):

$$\forall q \in Q. \forall a \in \Sigma. \delta(q, a) \in Q$$

Nondeterministic Finite Automata (NFA):

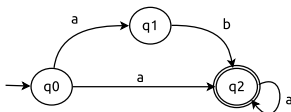
$$\forall q \in Q. \forall a \in \Sigma \cup \{\epsilon\}. \delta(q, a) \subseteq Q$$

A **dead** state is a state without outgoing transitions. We sometimes omit dead states when they are not final.

Finite Automata

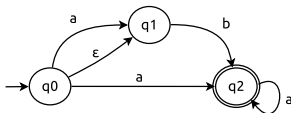
A word is **accepted** by a FA if it labels a path from the start state to a final state. The language accepted by a FA is the set of words it accepts.

Example - Consider the following FA:



It accepts: a; ab; aba; abaa; ...

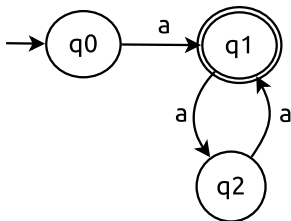
Example - Consider the following FA:



It accepts: a; b; ab; aba; ba; abaa; baa; ...

Finite Automata

What is the language accepted by:



This DFA accepts $\{a, aaa, aaaaa, \dots\}$

Finite Automata

It is easy to check whether a word is accepted or rejected by a (deterministic) FA

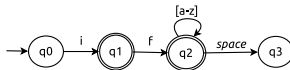
A lexer has to find the longest match. **How can we do that?**

We can keep track of the last time the FA was in a final state, i.e.:

- ▶ the state q
- ▶ and the word w read at that time

When a dead state is reached or the input is entirely read and the current state is a final state, w is returned.

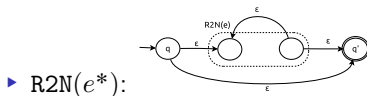
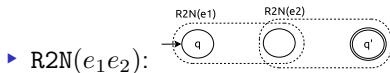
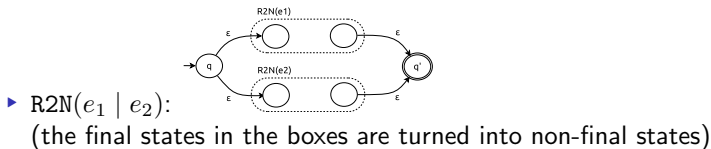
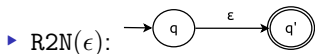
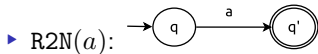
For example:



- ▶ When reading *ifif*, we record: $(q1, i); (q2, if); (q2, ifi); (q2, ifif)$ and return *ifif*
- ▶ When reading *if_*, we record: $(q1, i); (q2, if)$ and return *if*

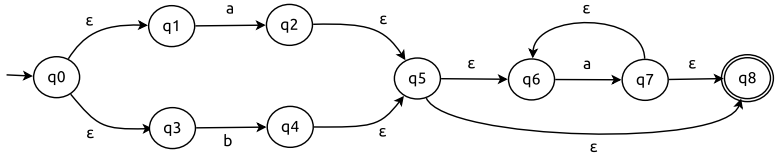
RE $\rightarrow$ NFA

Given a regular expression describing a language, we can convert it to an NFA as follows (essentially Thompson's algorithm):



RE $\rightarrow$ NFA

Example: $R2N((a \mid b)a^*)?$



NFA $\rightarrow$ DFA

Current state:

- ▶ Converting a RE to a NFA is easy
- ▶ Implementing an NFA as a program is harder than a DFA (we have to “guess” what transition to take)

We can **convert an NFA M to a DFA M'** (written $N2D(M)$):

- ▶ **input:** NFA $M = \langle Q, \Sigma, \delta, q_0, F \rangle$
- ▶ **output:** DFA $M' = \langle Q', \Sigma', \delta', q'_0, F' \rangle$

where $\Sigma' = \Sigma$ and a set in Q' is a set of states in Q , i.e.,
 $Q' = \{S \mid S \subseteq Q\}$

The other components rely on the following **closure** operator:

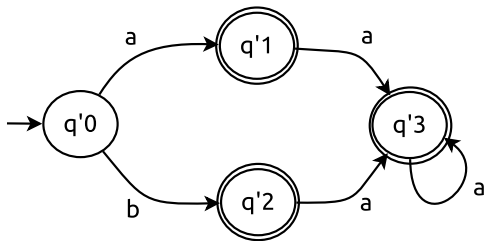
$$\text{closure}(S) = \{q' \in Q \mid \exists q \in S. q \xrightarrow{\delta, \epsilon} q'\}$$

where $q \xrightarrow{\delta, \epsilon} q'$ is true either if $q' = q$ or $(q'' \in \delta(q, \epsilon) \text{ and } q'' \xrightarrow{\delta, \epsilon} q')$

- ▶ $\delta'(S, a) = \text{closure}(\{q' \in \delta(q, a) \mid q \in S\})$
- ▶ $q'_0 = \text{closure}(\{q_0\})$
- ▶ $F' = \{S \in Q' \mid S \cap F \neq \emptyset\}$

NFA $\rightarrow$ DFA

What DFA do we obtain from the NFA we obtained from $\text{R2N}((a \mid b)a^*)$?



where $q'_0 = \{q_0, q_1, q_3\}$; $q'_1 = \{q_2, q_5, q_6, q_8\}$; $q'_2 = \{q_4, q_5, q_6, q_8\}$;
 $q'_3 = \{q_6, q_7, q_8\}$

Note that this DFA is not optimal. It could be further **minimized** by merging the final states.

Building a Lexer

Goal: Given an ordered list of REs $e_1, \dots, e_n$ (capturing lexemes), and a word w :

- ▶ find the longest prefix of w that is accepted by $N2D(R2N(e_1 \mid \dots \mid e_n))$
- ▶ repeat the process until w is read

I.e., find a list $(k_1, w_1), \dots, (k_m, w_m)$ such that $w = w_1 \dots w_m$ and $\forall i \leq m. \exists j \leq n. w_i$ is accepted by $N2D(R2N(e_j)) \wedge k_i = j$

Example: let us apply this to

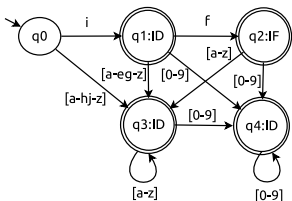
if	<i>if</i>
identifier	$[a - z][a - z]^*[0 - 9]^*$
...	

Building a Lexer

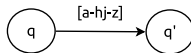
Regular expressions specifying our lexemes:

if *if*
identifier $[a-z][a-z]^*[0-9]^*$
...

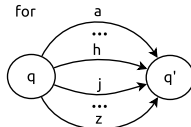
Minimized DFA:



we write



for



Note:

- ▶ Final states are also annotated with token names
- ▶ Token values are the words read
- ▶ *if* is accepted as an **IF** and not an **ID**
- ▶ We have to return the longest matches

Conclusion

What did we cover today?

- ▶ Regular expressions
- ▶ Finite Automata
- ▶ Regular expressions to NFAs
- ▶ NFAs to DFAs
- ▶ Building a lexical analyzer

Next time?

- ▶ Parsing